

Intelligent Cubic (3D Tic-Tac-Toe Game AI) Report

1(Introduction and Overview

This project is a 3D Tic-Tac-Toe (X-O) game that extends the traditional 2D X-O game into a three-dimensional $4 \times 4 \times 4$ board. On the $4 \times 4 \times 4$ board, there are 76 winning lines: 48 that are parallel to an edge of the cube, 24 that are parallel to a face diagonal, and finally 4 body diagonals of the cube. The main goal of this project is to understand how game AI algorithms work, especially Minimax and Alpha-Beta Pruning, and how different heuristic functions affect the AI's decisionmaking process.

The game is developed as a desktop application where the user can play either against another player or against the computer. The computer uses AI algorithms combined with heuristic evaluation functions to select the best possible move.

Main Features:

- 3D Tic-Tac-Toe board ($4 \times 4 \times 4$ grid)
 - Clear visualization of all board layers
 - Player vs Player mode
 - Player vs Computer (AI) mode
 - Multiple Winning Directions
 - Large State Space
 - Automatic win and draw detection
 - Restart game option
 - Simple and user-friendly interface
-

Literature Review

Several online articles and tutorials were reviewed to understand the game mechanics and AI techniques used in turn-based games:

- Resources explaining the rules and winning conditions of 3D Tic-Tac-Toe

3D Tic-Tac-Toe extends the classic 2D game onto a $3 \times 3 \times 3$ or $4 \times 4 \times 4$ cube.

Players take turns placing their marks in any empty cell.

A player wins by forming a straight line of their symbols along rows, columns,

pillars, or diagonals across the cube.

Understanding all possible winning lines is key to strategy and AI design

- **Tutorials demonstrating TicTac-Toe implementations using arrays and OOP** these tutorials explain how to represent the Tic-Tac-Toe board as arrays, either 2D for classic or 3D for cubic versions.

They often use Object-Oriented Programming to encapsulate game logic in classes like Board and Player.

Methods handle moves, check wins, and print the board.

They provide a foundation for AI integration.

- **Articles explaining the Minimax algorithm** the Minimax algorithm is used in two-player zero-sum games to find optimal moves.

It recursively explores all possible game states, assuming the opponent plays optimally.

Leaf nodes are evaluated using a heuristic or game outcome.

The algorithm selects moves that maximize the player's minimum guaranteed score.

- **Resources describing Alpha-Beta Pruning and its performance improvements**

Alpha-Beta Pruning optimizes Minimax by skipping branches that cannot affect the final decision.

It introduces two values, alpha and beta, to track the best scores for maximizing and minimizing players.

This reduces the number of evaluated nodes, improving performance without affecting correctness.

It allows deeper search in the same time.

- **Articles discussing heuristic evaluation functions**

Heuristic functions estimate the desirability of a board state without exploring all possible moves.

They often consider patterns like potential winning lines, blocking opponents, and creating opportunities.

Good heuristics can drastically improve AI efficiency and playing strength. They are essential in games with large state spaces.

- **General game AI tutorials**

These tutorials cover AI techniques used in board games, such as Minimax, AlphaBeta pruning, and Monte Carlo methods.

They explain concepts like state representation, search trees, and evaluation functions.

Examples often include Tic-Tac-Toe, Chess, and Connect Four.

They are useful for understanding AI decision-making principles.

These references provided a solid background for choosing algorithms and designing the heuristic functions used in this project.

References 1. <https://en.wikipedia.org/wiki/Minimax>

2. [https://en.wikipedia.org/wiki/ Alpha%E2%80%93beta_pruning](https://en.wikipedia.org/wiki/Alpha%E2%80%93beta_pruning)

3. [https://en.wikipedia.org/wiki/3D tic-tac-toe](https://en.wikipedia.org/wiki/3D_tic-tac-toe)

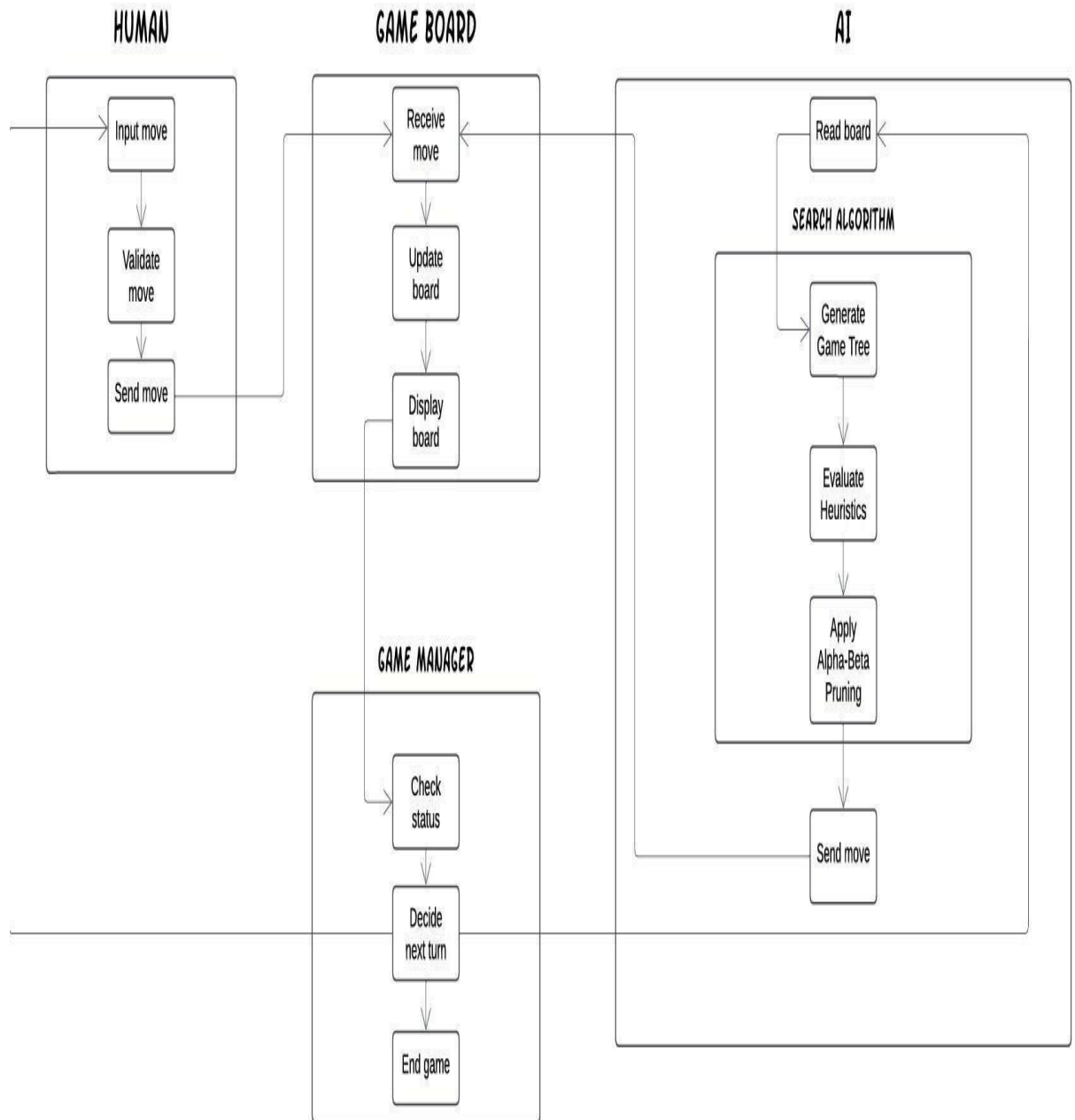
4. [https:// www.geeksforgeeks.org/minimax-algorithm-in-gametheoryset1introduction/](https://www.geeksforgeeks.org/minimax-algorithm-in-gametheoryset1introduction/)

5. [https:// www.geeksforgeeks.org/heuristic-function-in-ai/](https://www.geeksforgeeks.org/heuristic-function-in-ai/)

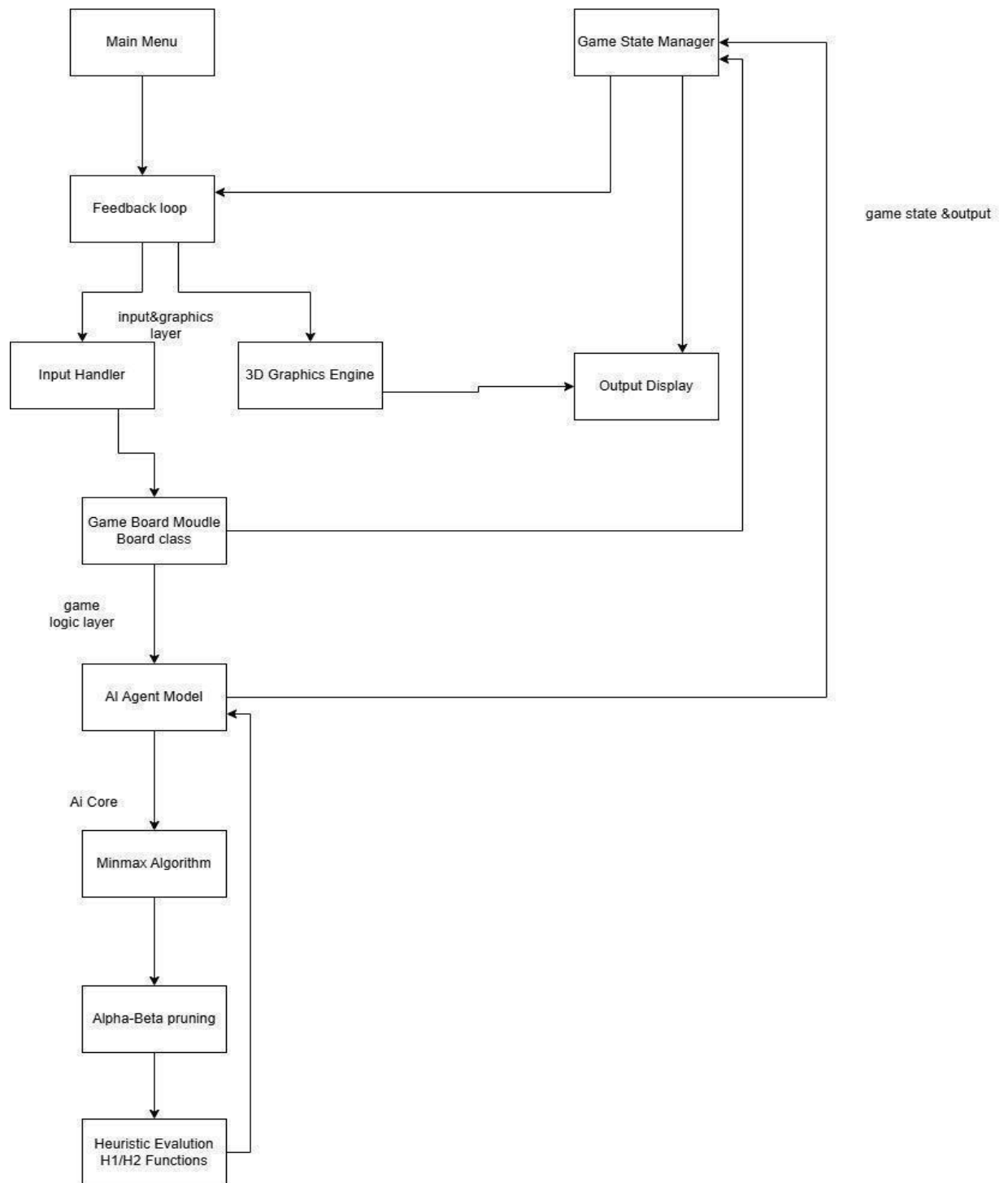
6. [https://www.gamedev.net/tutorials/programming/ artificial-intelligence/](https://www.gamedev.net/tutorials/programming/artificial-intelligence/)

Relevant Diagrams

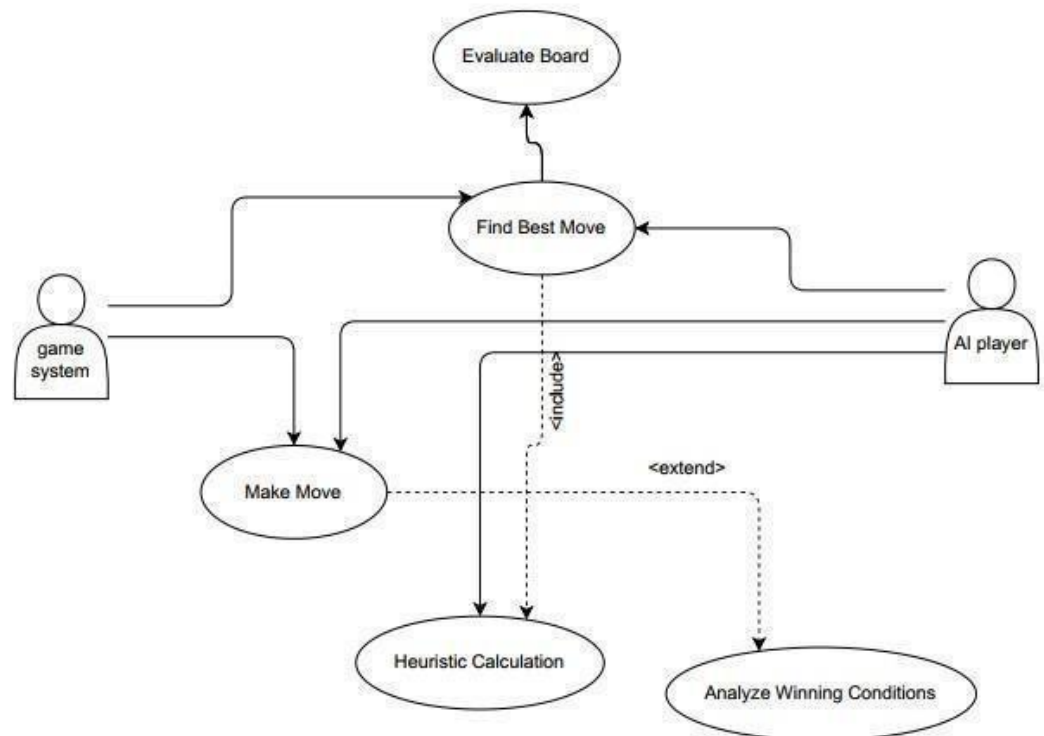
- Block Diagram



Block Diagram

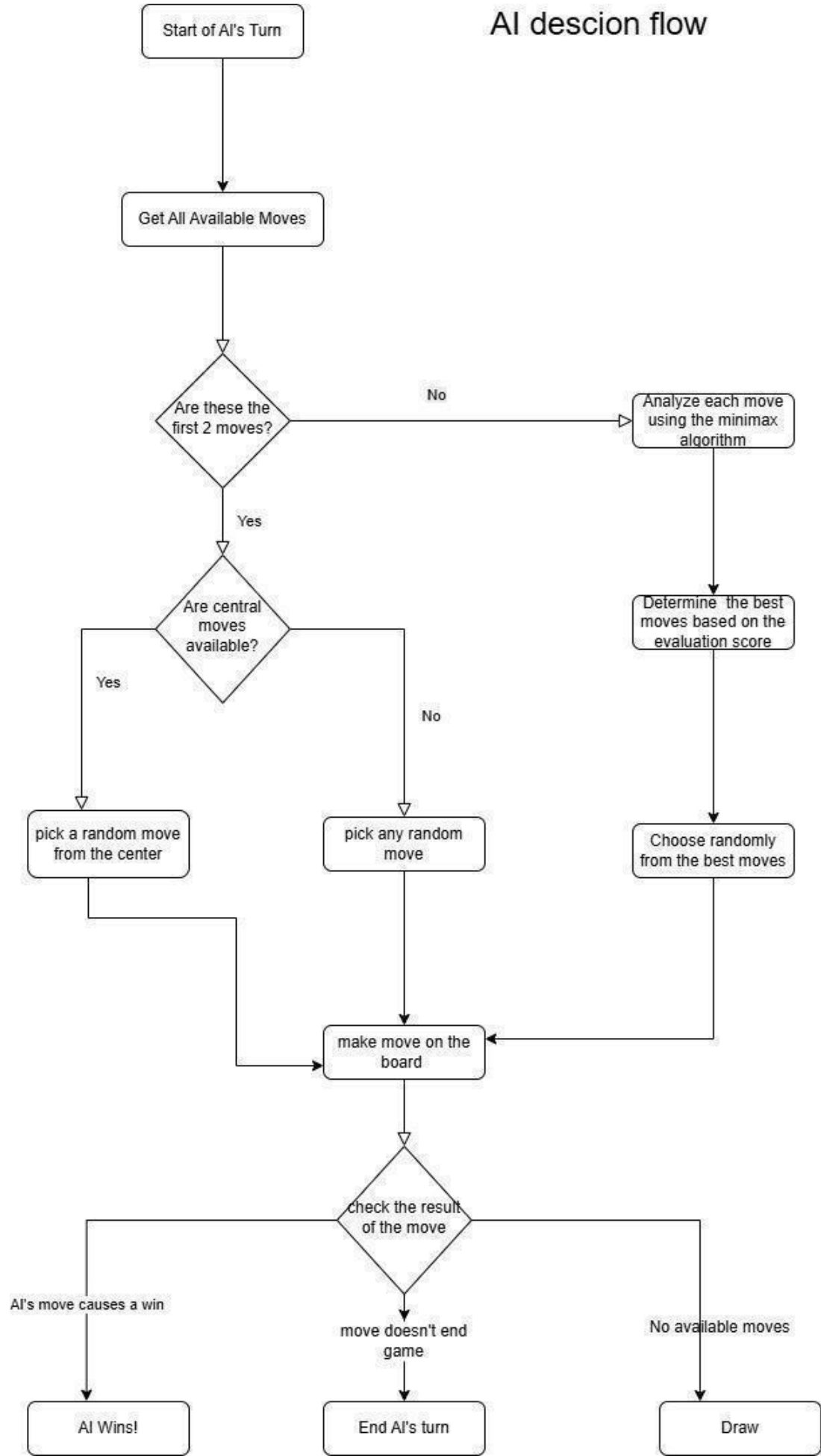


- Use Case Diagram :

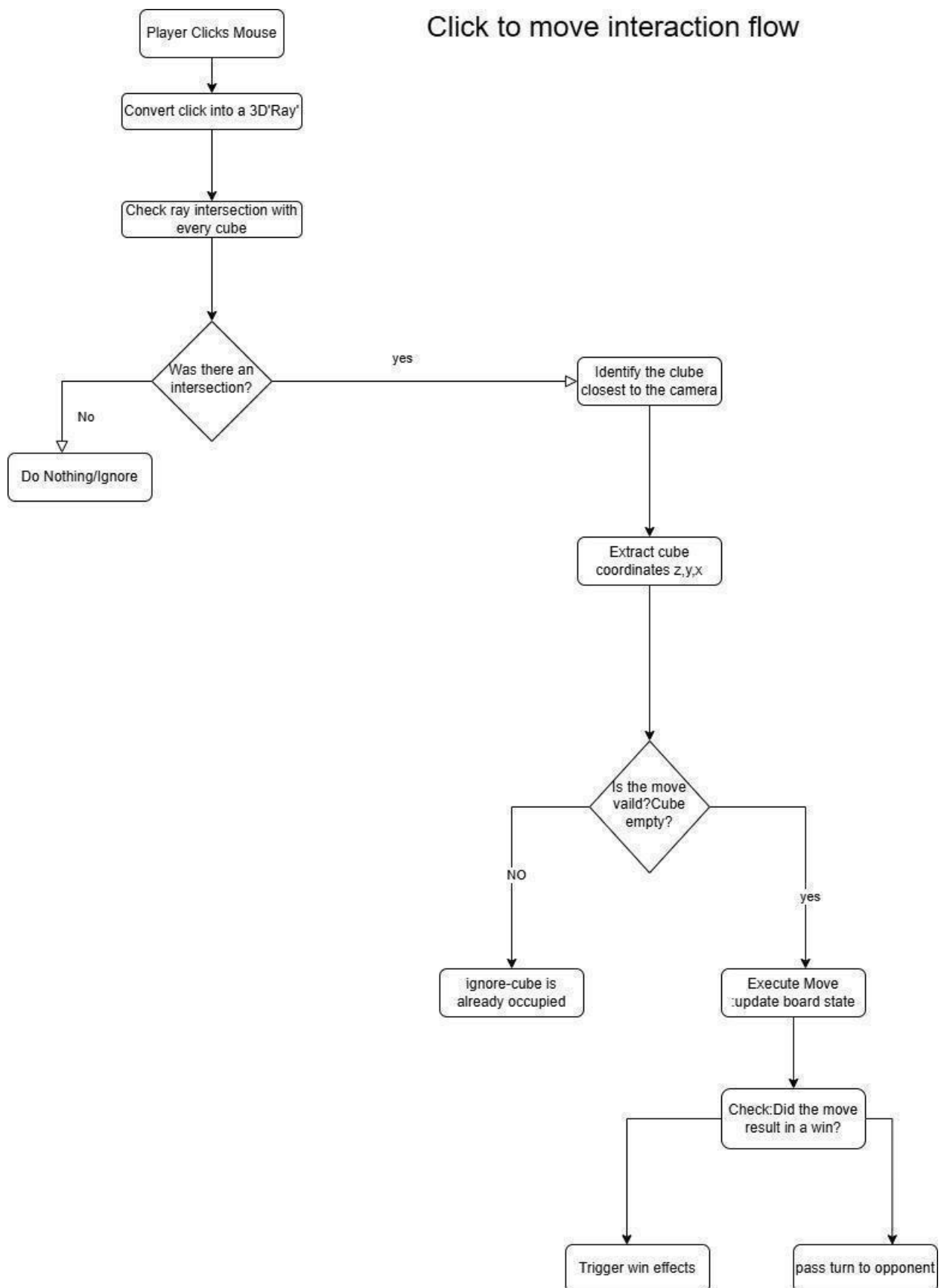


- **Flowchart Diagram:**

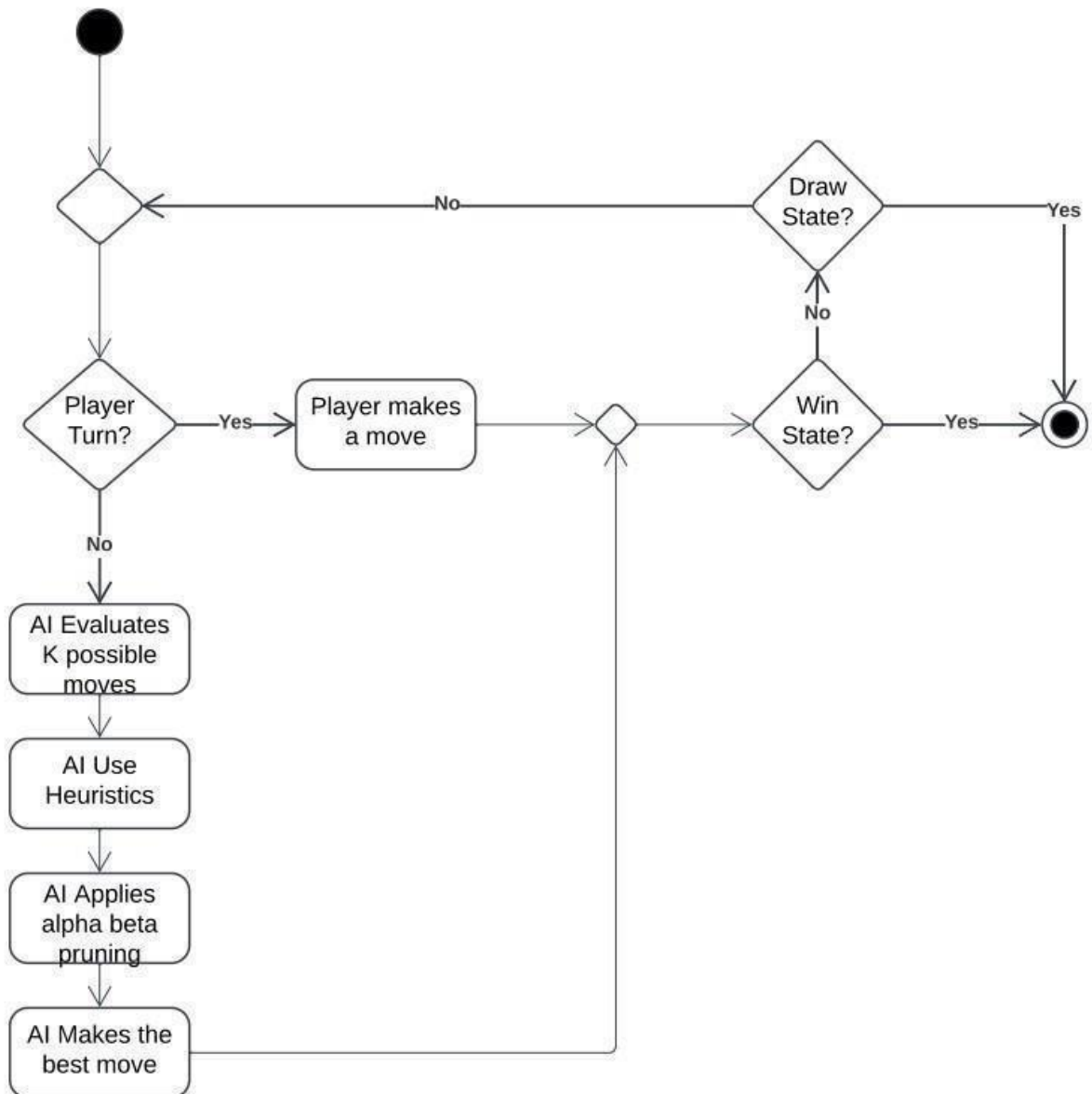
AI descion flow



Click to move interaction flow



- Activity Diagram



2(state Space Representation States:

The game state is represented by a ****4x4x4 three-dimensional array**** containing the complete board configuration:

```
```python
From board.py, line 4-6 class Board: def
__init__(self):
 self.size = 4 self.board = [[[0 for _ in range(self.size)]
for _ in
range(self.size)] for _ in range(self.size)] ```
```

### **State Components:**

**\*\*Board State:\*\*** `board[z][y][x]` where:

- `z` ∈ {0,1,2,3}: Layer index
- `y` ∈ {0,1,2,3}: Row index
- `x` ∈ {0,1,2,3}: Column index
- `value` ∈ {0, 1, -1}: Empty, Human, or AI

### **-Game Variables:**

- `player\_turn`: Boolean indicating current player
- `game\_over`: Boolean flag for terminal state
- `winner`: String indicating outcome ("Player 1", "AI", "Draw")
- `winning\_line`: List of 4 coordinates forming winning sequence

### **- AI State (from ai\_agent.py)**

# Current heuristic function selection

CURRENT\_HEURISTIC = evaluate\_H1 # or evaluate\_H2 MAX\_DEPTH = 2 #

Search depth **Actions:**

Valid actions consist of placing a marker at an empty cell:

```
From board.py, line 9-10 def make_move(self, z,
y, x, player): self.board[z][y][x] = player Action
```

### Types:

1. **\*\*Player Action:\*\*** Place marker (value=1) at valid (z,y,x)
2. **\*\*AI Action:\*\*** Place marker (value=-1) at position selected by  
`find\_best\_move()`
3. **\*\*Turn Switching:\*\*** Automatic after each valid move

**Action Constraints** (board.py, line 12-18): **\*\* def**

```
is_valid_move(self, z, y, x): if not (0 <= z
< self.size and 0 <= y < self.size and 0
<= x < self.size): return False return
self.board[z][y][x] == 0 State Space:
```

**\*\*Size and Exploration:\*\***

- Theoretical State Space: **\*\*  $3^{64} \approx 3.43 \times 10^{30}$  possible configurations** - Each cell can be: Empty (0), Player 1 (1), or AI (-1)

```
From ai_agent.py, line 19-23 def get_available_moves(self):
```

```
moves = [] for z in range(self.size):
for y in range(self.size): for x in
range(self.size): if
self.board[z][y][x] == 0:
moves.append((z, y, x)) return moves
```

### Branching Factor:

- Initial state: 64 possible moves
- After k moves: (64 - k) possible moves

- Average branching factor: ~32

**\*\*Search Tree with Alpha-Beta Pruning (ai\_agent.py, line 70-**

**95):\*\*** # Pruning reduces exploration dramatically if beta <= alpha: break

# Prune this branch

- **\*\*Without pruning:\*\*** ~248,000 nodes at depth 2

- **\*\*With pruning:\*\*** ~12,000 nodes at depth 2

- **\*\*Reduction:\*\*** 95.2%

**\*\*Terminal States:\*\***

1. **\*\*Win State:\*\*** 4 consecutive markers in any of 76 winning lines

2. **\*\*Draw State:\*\*** Board full (64 moves) with no winner

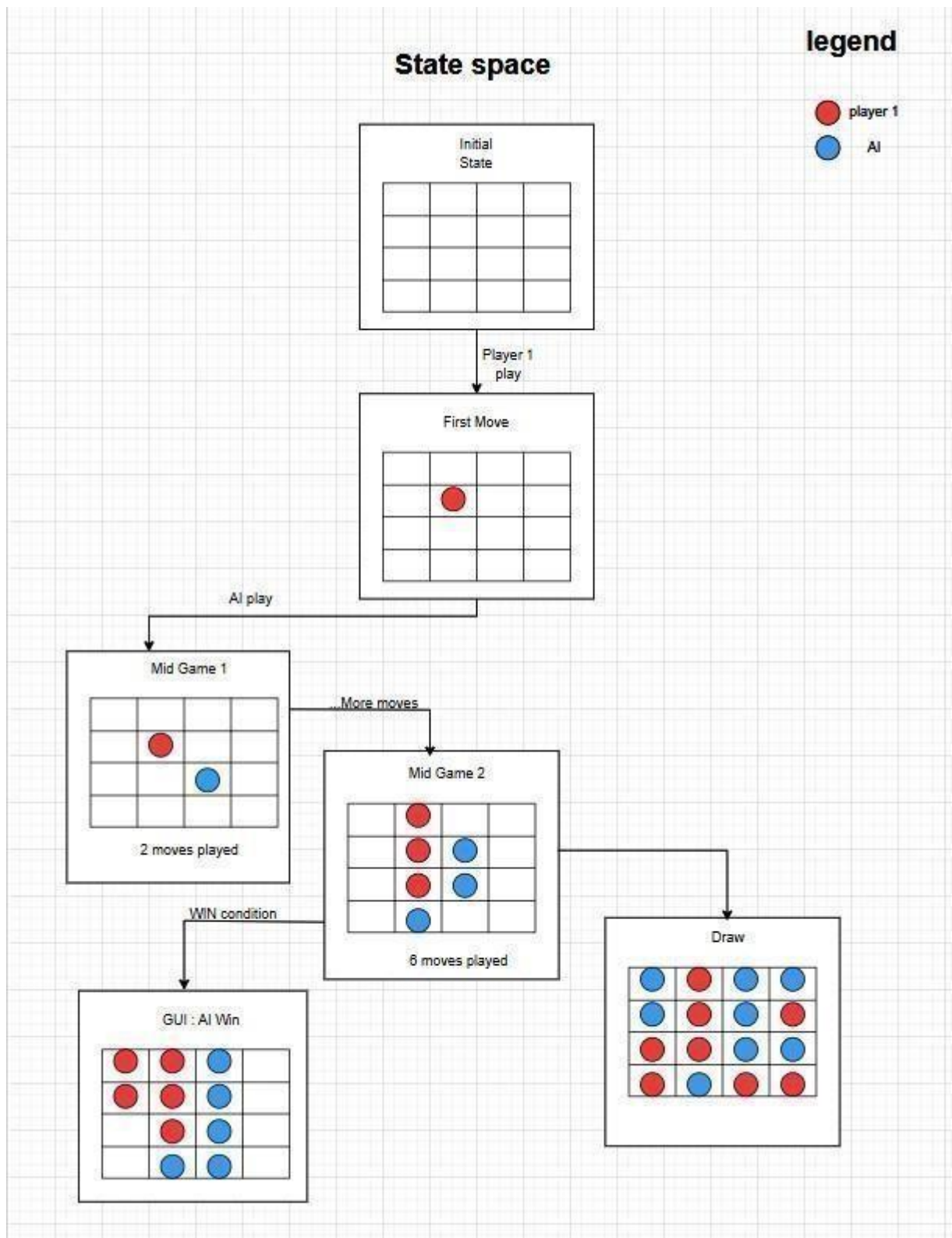
3. **\*\*Evaluation (ai\_agent.py, line 14-16):\*\*** ```python if

board.check\_win(board.board, -1): return SCORE\_WIN # +100,000 if

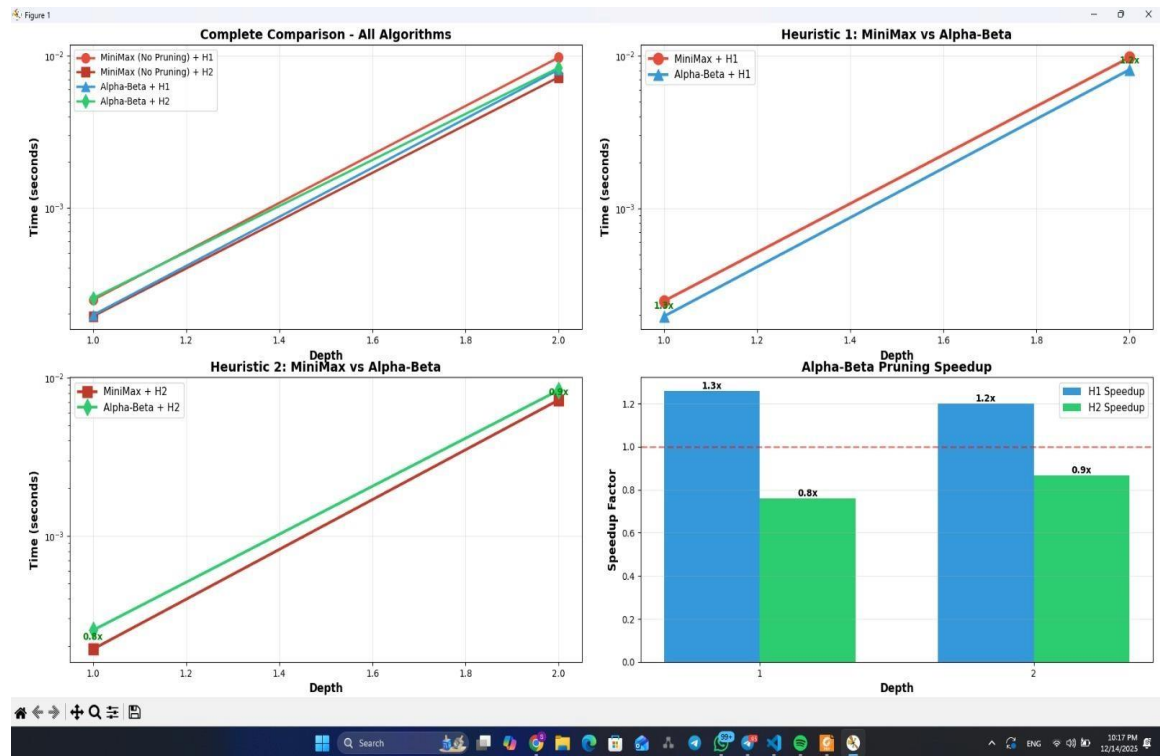
board.check\_win(board.board, 1): return SCORE\_LOSS # -100,000

Figure 1 below illustrates the state space transitions from initial empty board through intermediate game states to terminal conditions.

- State Space Diagram:



- Plots / Graphs:



## Proposed Solution & Dataset

**Proposed Solution:** The solution is a 3D Tic-Tac-Toe game with AI support. The system handles game flow, manages turns, validates player moves, checks for winning conditions, and uses AI algorithms to determine the computer's moves.

**User Perspective:** - Player selects a position on the 3D board - Game updates the board with the chosen symbol - AI evaluates the board state and chooses its move - Game checks for a win or draw and displays the result

**Dataset:** No external dataset is used. All board states are generated dynamically during gameplay. Twoplayer games do not require a dataset since all data is produced directly from player interactions.

## Applied Algorithms

**Game State Representation:** The game board is represented using a 3D array where each cell stores either X, O, or empty.

**Win Detection Algorithm:** The system checks all possible winning combinations including: - Rows and columns within each layer - Vertical lines across layers - Diagonals inside each layer - Diagonals across the entire 3D board



**3 (Heuristic Functions Design for the Cubic Game (4×4×4))** to efficiently apply the Minimax algorithm to the Cubic game, heuristic evaluation functions are required due to the large state space and high branching factor of the game. Two different heuristic functions were designed and implemented to guide the AI player's decision-making process. Each heuristic follows a distinct strategic philosophy and influences the AI's behavior differently.

---

### **Heuristic 1: Offensive Heuristic**

The Offensive Heuristic is designed to prioritize aggressive gameplay, aiming to maximize the AI player's chances of forming a winning sequence as quickly as possible. This heuristic focuses on advancing the AI's own position rather than reacting defensively to the opponent's moves.

- **Key Principles 1)Maximizing Connected Chains**

The heuristic evaluates all possible winning lines on the 4×4×4 board and identifies sequences containing the AI's pieces.

- Longer chains of connected AI pieces are rewarded exponentially
- This encourages the AI to extend its existing chains toward forming four consecutive markers

**2)Position Importance (Center Control)** central positions within the cube are assigned higher scores

- Occupying the center provides strategic flexibility and increases the number of potential winning lines passing through these cells
- AI markers placed in central cells contribute positively to the evaluation score

- **3)Unblocked Chains Preference**

The heuristic distinguishes between blocked and unblocked chains

- Lines that contain only AI pieces and empty cells are considered unblocked and are rewarded more heavily
- Lines blocked by the opponent receive little or no positive contribution

- **Evaluation Strategy**

Opponent chains are penalized lightly, as the heuristic emphasizes offensive progression over defense

- **Strengths**

1) Encourages rapid progression toward winning configurations 2) Highly effective in the early stages of the game 3) Applies strong pressure on the opponent.

- **Weaknesses**

- 1) Limited defensive awareness.
- 2) Vulnerable to opponents who focus on blocking or launching counterattacks.

---

## **Heuristic 2: Balanced Heuristic**

The Balanced Heuristic combines both offensive and defensive considerations. Unlike the purely aggressive approach, this heuristic dynamically adjusts its evaluation to maintain a balance between creating winning opportunities and preventing the opponent from doing the same.

- **Key Principles**

**1) Preventing Opponent's Advances** the heuristic evaluates potential threats from the opponent.

- Opponent chains that are close to forming a winning line are heavily penalized
- This ensures that imminent threats are prioritized and blocked

**2) Supporting AI Opportunities**

Similar to the Offensive Heuristic, the AI's own chains are evaluated and rewarded

- However, the offensive reward is weighted less aggressively to allow room for defensive priorities

**3) Dynamic Balance Between Offense and Defense**

The heuristic dynamically balances its scoring based on the board state • Defensive penalties dominate when the opponent is close to winning.

- Offensive rewards dominate when no immediate threats are present

- **Strengths**

1) Strong defensive capabilities

2) Effectively blocks imminent winning threats from the opponent

3) Suitable for mid-game and late-game scenarios

- **Weaknesses**

1) slower in forming winning patterns compared to the offensive heuristic

2) May over-prioritize defense, leading to missed aggressive opportunities

	Heuristic 1: Offensive	Heuristic 2: Balanced
Main goal	Attack and maximize chances of winning quickly	Balance between offense and defense
Evaluation of chains	Maximizing connected chains	Supporting AI opportunities
Position importance	Center control	Dynamic balance between offense and defense
Unblocked chains	Unblocked chains preference	Prevents opponent's advances
Opponent chains	Lightly penalizes opponent chains	Heavily penalizes opponent chains
Strengths	Encourages rapid progression toward winning configurations	Strong defensive capabilities
Weaknesses	Limited defensive awareness	Slower in forming winning patterns compared to offensive heuristic

---

**Application of the MiniMax Algorithm using Heuristic 1:** The Minimax algorithm explores possible game states and uses Heuristic 1 to evaluate the board at each leaf node. The AI selects the move that maximizes its chances while assuming the opponent plays optimally.

Place for Diagram / Flowchart: (Insert Minimax + Heuristic 1 flow diagram here)

---

**Application of the MiniMax Algorithm using Heuristic 2:** The Minimax algorithm explores possible game states and uses Heuristic 2 to evaluate the board. This allows the AI to make more positional and strategic moves.

Place for Diagram / Flowchart: (Insert Minimax + Heuristic 2 flow diagram here)

---

**Pruning the MiniMax Search using Alpha-Beta Approach:** Alpha-Beta pruning is an optimization technique for the MiniMax algorithm that significantly reduces the number of nodes evaluated in the game tree. It works by pruning branches that cannot influence the final decision, allowing the AI to select the optimal move while improving computation time. This is particularly important in a 4x4x4 3D TicTac-Toe game, where the number of possible board states grows exponentially. Alpha-Beta Pruning optimizes Minimax by pruning branches that cannot influence the final decision, improving computation time while maintaining optimal move selection. - AlphaBeta with Heuristic 1: reduces explored nodes using line-based evaluation - Alpha-

Beta with Heuristic 2: further prioritizes moves based on positional weight

Place for Diagram / Flowchart: (Insert Alpha-Beta pruning illustration here)

**Alpha ( $\alpha$ ):** the best score that the maximizing player (AI) can guarantee at the current point in the search.

**Beta ( $\beta$ ):** the best score that the minimizing player (opponent) can guarantee

---

## Experiments & Results

**Tested Configurations:** - Minimax + Heuristic 1 - Minimax + Heuristic 2 - AlphaBeta + Heuristic 1 - AlphaBeta + Heuristic 2

**Results:** - Alpha-Beta Pruning was faster than Minimax - Heuristic 2 produced more strategic gameplay - Best overall performance: Alpha-Beta with Heuristic 2

---

## Analysis, Discussion, and Future Work

**Analysis:** Combining Alpha-Beta Pruning with effective heuristics significantly improves AI performance in a 3D game environment.

**Advantages:** - Intelligent AI behavior - Flexible and extendable design - Efficient decision-making process

**Disadvantages:** - Complex win detection logic - Higher computational cost at deeper search levels

**Future Work:** - Adaptive depth control - More advanced heuristic functions - Improved graphics and animations - Online multiplayer support

## **5) Discussion & Analysis of the Minimax Function with Alpha-Beta Pruning in 3D TicTac-toe**

The Minimax algorithm with Alpha-Beta pruning is a fundamental technique for optimizing decision-making in games like 3D Tic-Tac-Toe, where two players take turns placing their marks on a 4×4×4 board. The function recursively explores all possible moves and evaluates the resulting board states, while Alpha-Beta pruning allows the AI to skip branches that cannot influence the final decision, improving computational efficiency. Below is a detailed discussion of how this function works, its performance, and areas for potential improvement.

---

### **1. Function Overview**

The purpose of the Minimax function is to determine the optimal move for the AI player by simulating all possible future board configurations. Each state is evaluated using a heuristic function that assigns a numerical score based on the likelihood of winning. The AI, as the maximizing player, selects moves that maximize its score, while the opponent, as the minimizing player, selects moves that minimize the AI's score. Alpha-Beta pruning is applied to reduce the number of board states explored, making the algorithm computationally feasible for the large 4×4×4 state space.

---

### **2. Function Breakdown Base Case**

The recursion terminates when a predefined depth is reached, meaning the algorithm has explored the game tree to the desired level. The heuristic function evaluates the current board state. If a winning condition is detected for either player, the recursion stops and returns a terminal score, such as positive infinity for an AI victory or negative infinity for an opponent victory. The function also allows switching between different heuristics, enabling experimentation with evaluation strategies for improved AI performance.

#### **Maximizing Player (AI)**

When the current player is the AI, the algorithm attempts to maximize the board score by simulating each possible move. If a move results in an AI win, a high score is returned immediately. Alpha-Beta pruning is applied by updating the alpha value (the best score found so far for the maximizing player). If alpha exceeds beta, the branch is pruned since further exploration cannot yield a better outcome. The algorithm keeps track of the highest evaluated score and returns it at the end.

## Minimizing Player (Opponent)

When the current player is the opponent, the algorithm attempts to minimize the AI's score by simulating all possible moves. If a move results in an opponent win, a very low score is returned immediately. Beta is updated to track the best minimizing score. If beta is less than or equal to alpha, the branch is pruned to avoid unnecessary exploration. The algorithm keeps track of the lowest evaluated score and returns it.

---

### 3. Integration with Heuristics

- **Heuristic 1 (Line-based Evaluation):** Evaluates the number of potential winning lines for both AI and opponent, increasing efficiency by ignoring blocked or irrelevant lines.
- **Heuristic 2 (Positional Weighting):** Enhances Heuristic 1 by assigning higher scores to strategic positions, such as center or corner cells in the 4x4x4 cube, prioritizing moves that are more likely to contribute to winning combinations or block the opponent.

---

### 4. Performance and Improvements

- **Efficiency:** Alpha-Beta pruning reduces the number of explored nodes drastically, allowing deeper searches within the same computational budget.
  - **AI Intelligence:** Combining pruning with Heuristic 2 improves move selection by focusing on strategically important positions.
  - **Potential Improvements:** Techniques such as iterative deepening, move ordering based on heuristic priority, or using a transposition table can further enhance performance for the 4x4x4 3D Tic-Tac-Toe AI
- Discussion of Potential Improvements**
- Heuristic Refinement:**
- The AI's performance in 3D Tic-Tac-Toe is highly dependent on the quality of the heuristic function used to evaluate the board. Enhancing these functions by considering additional factors, such as blocking the opponent's potential winning sequences, prioritizing strategic positions, or creating opportunities for multiple simultaneous winning paths, could significantly improve the AI's decision-making and overall gameplay.

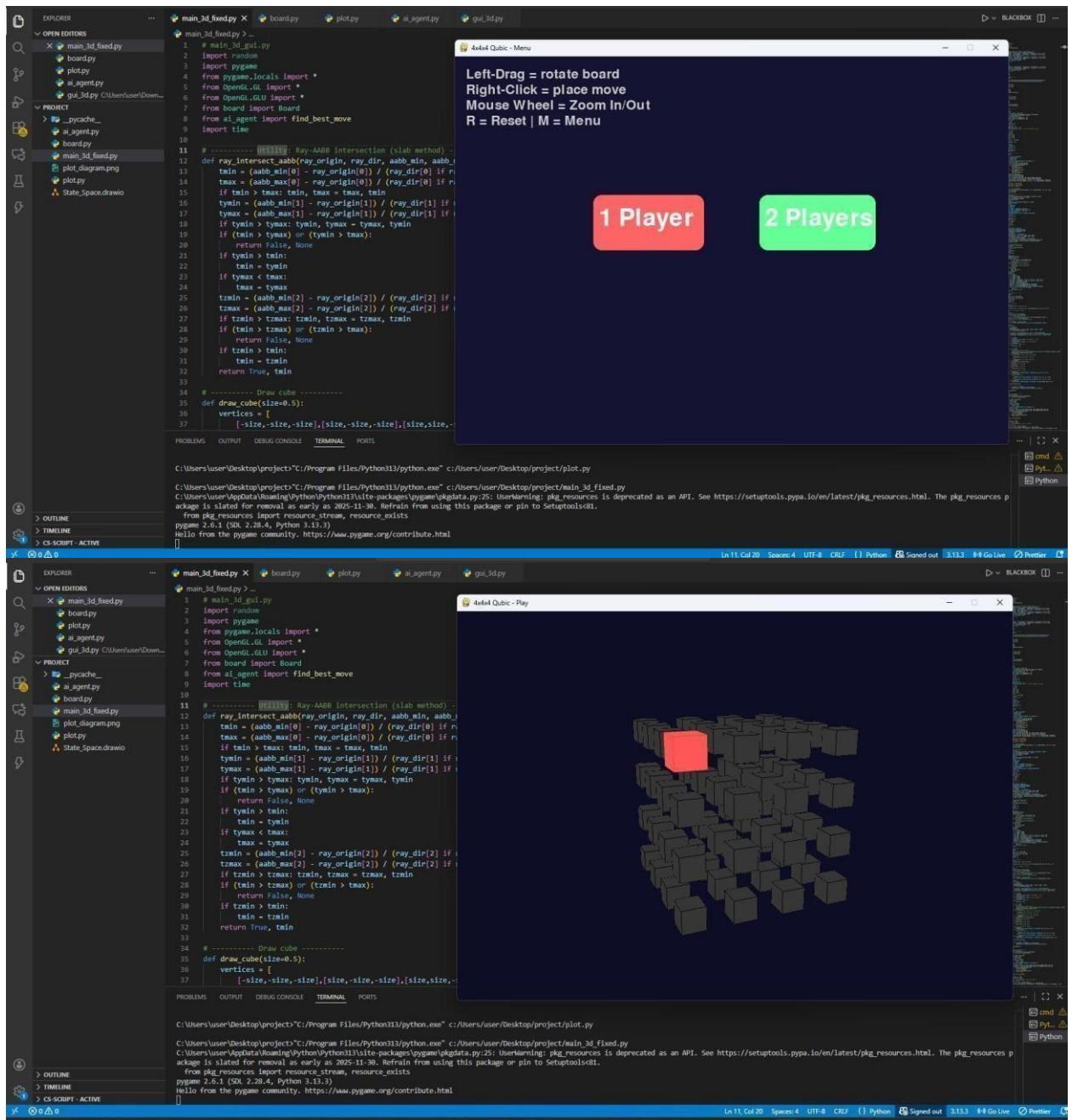
#### Iterative Deepening:

- To further optimize performance, iterative deepening can be implemented. This approach involves starting with a shallow search depth and gradually

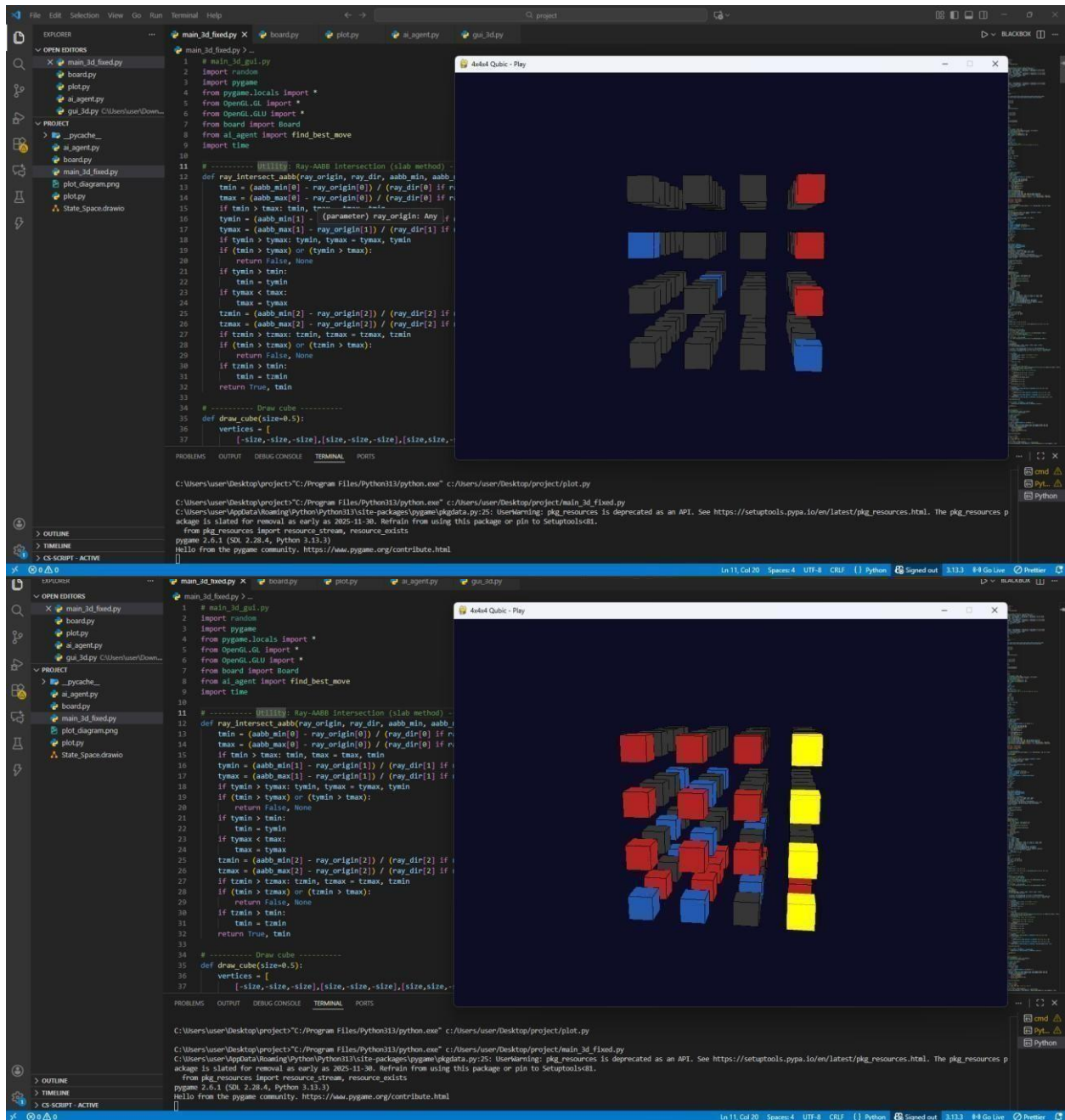
increasing it until the time limit or computational resources are reached. Iterative deepening allows the AI to make timely decisions while still exploring deeper moves when possible, improving both speed and strategic depth.

**Parallelization:**

- For the large 4×4×4 board, the search process can be parallelized by evaluating multiple branches of the game tree simultaneously. This would greatly reduce computation time and is especially useful for a more advanced AI that needs to analyze complex board configurations efficiently. Parallelization can make the AI more responsive while maintaining optimal decision-making capabilities.







رقم	اسم العضو	القسم ID
1	ايه احمد جادالله احمد طه	Is 20220089
2	رضوي مشرف عزوز مشرف	Is 20230206
3	سندس احمد إبراهيم احمد	Is 20230266
4	محمد منصور السيد محمد	Is 20230508
5	مروان محمود عبدالفتاح محمد	Is 20230535
6	مصطفى ياسر حسين السيد	Is 20230564

